

I. Expliquer Docker

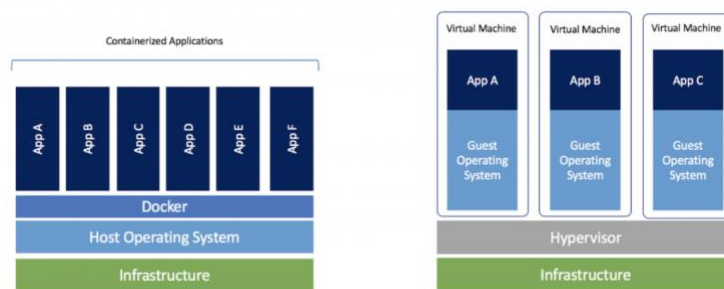
Docker est une solution logiciel de conteneurisation. On appelle ses applications des conteneurs. Il peut être apparenté à de l'Infrastructure as a Code (IaC). Docker n'est pas une solution de virtualisation, mais permet de traiter les conteneurs comme des machines virtuelles très légères et modulaires.

Un conteneur Docker contient un système d'exploitation et l'application elle-même.

C'est une machine virtuelle Linux qui exécute le Docker engine.

Les deux sont bien des types de virtualisation cependant ils sont différents de par la façon dont ils utilisent les ressources de la machine hôte.

Quand on crée une machine virtuelle, il faut allouer une certaine quantité de la mémoire de l'hôte à la machine virtuelle. L'hyperviseur (programme qui gère la virtualisation) établit donc une séparation entre les ressources des différentes VM ainsi que celles de l'hôte. Alors que la création de conteneur ne nécessite pas d'allocation de ressource physique. Ceci est possible parce que tous les conteneurs utilisent directement le Kernel de l'hôte.



II. Installer Docker

III. Lancer Docker pour un serveur apache

Créer un conteneur apache dans docker :

```
docker run -dit --name apache -p 8080:80 httpd:2.4
```

Lancer un navigateur avec comme lien : <http://localhost:8080>

On obtiendra la réponse de notre serveur apache contenu dans notre conteneur (it works).

Avec cette méthode, nous pouvons lancer un site web, mais nous ne pouvons pas changer son contenu.

Maintenant lorsque nous voulons héberger un site/application dans le conteneur, nous devons disposer de son dossier (on peut télécharger gratuitement un exemple via le site : <https://www.free-css.com/>

Une fois le site téléchargé, vous pouvez le dézipper et lancer un terminal à partir de ce dossier ou simplement vous positionnez dans le dossier via ligne de commande (`cd youssoukasse/Desktop/` `templatemo_560_astro_motion/`).

templatemo_560_astro_motion est site web que nous comptons héberger.

Maintenant pour demander au conteneur d'utiliser ce dossier, on peut créer un nouveau volume et faire le mapping avec notre conteneur. Le dossier étant accessible sur la machine hôte, son contenu peut être modifié.

L'option `-v` comporte deux parties : `-v "$PWD":/usr/local/apache2/htdocs/`

`$PWD` : représente le répertoire courant de la machine qui héberge le conteneur (donc accessible);

`/usr/local/apache2/htdocs/` : représente le répertoire à l'intérieur du conteneur (qui sera mappé avec le précédent répertoire).

On peut maintenant démarrer un nouveau conteneur (`apache2`). Mais nous pouvons arrêter le précédent(`apache`) pour libérer le port.

On peut maintenant redémarrer un nouveau conteneur en utilisant cette fois le volume (option `-v`).

```
docker run -dit --name apache2 -p 8080:80 -v "$PWD":/usr/local/apache2/htdocs/ httpd:2.4
```

Sous Windows, le répertoire courant("`$PWD`") sera remplacé par `${PWD}`).

On peut décider de créer notre propre image en utilisant Dockerfile.

Il faut dans ce cas, créer un nouveau dossier (`public-html`) dans (`templatemo_560_astro_motion`) et déplacer tous les fichiers dans ce nouveau dossier. Après cela, nous créons un nouveau fichier (sans extension) nommé (`Dockerfile`) et nous copions les lignes suivantes :

```
FROM httpd:2.4
COPY ./public-html/ /usr/local/apache2/htdocs/
```

Une fois enregistré, nous pouvons faire un build pour obtenir la nouvelle image :

```
docker build -t apache-img:1.0 .
```

Maintenant que l'image est créée, nous pouvons lancer un nouveau conteneur via cette image. Mais pour libérer le port, nous allons arrêter le précédent.

```
docker run -dit --name apacheContainer3 -p 8080:80 apache-img:1.0
```

On peut aussi interagir avec notre conteneur via ligne de commande bash avec :

```
docker exec -it apacheContainer3 bash
```

On peut aussi exécuter directement des commandes à partir du conteneur sans que cela se passe de manière interactive et sans se positionner sur le conteneur :

```
docker exec apacheContainer3 ping 8.8.8.8.
```

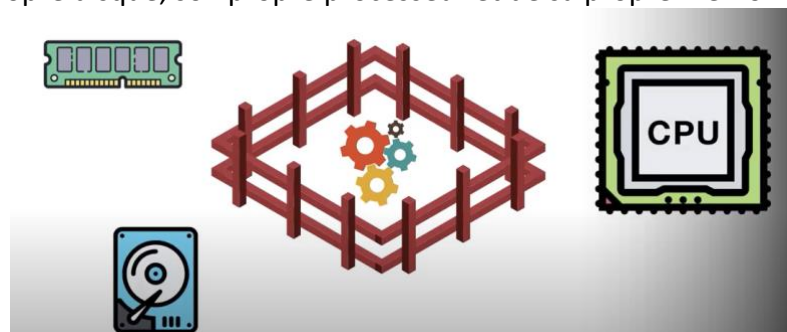
Mais il faudra vérifier que l'outil (ici ping est disponible dans le conteneur).

On peut également lister l'ensemble des conteneurs déployés sur une machine hôte avec la commande : `docker stats --all`

Tous les conteneurs (arrêtés ou pas) seront listés avec leurs statistiques en CPU, disque et mémoire.

IV. Fonctionnement d'une application Docker

Quand une application fonctionne dans un conteneur, elle se croit seule au monde. Elle a son propre disque, son propre processeur et de sa propre mémoire.

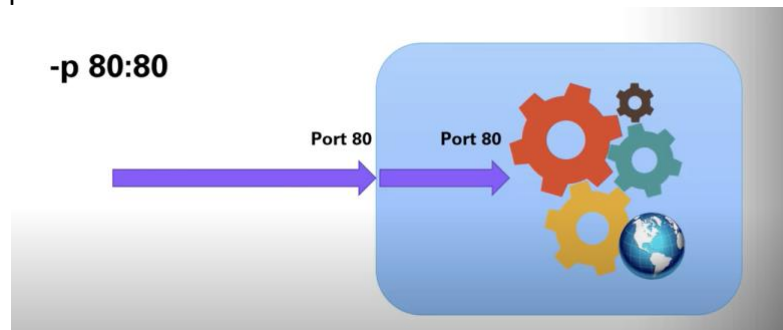


Pour communiquer avec le conteneur, on utilise des **mappings**. Les mappings sont des fonctionnalités qui permettent de créer des liens entre l'extérieur et l'intérieur du conteneur.



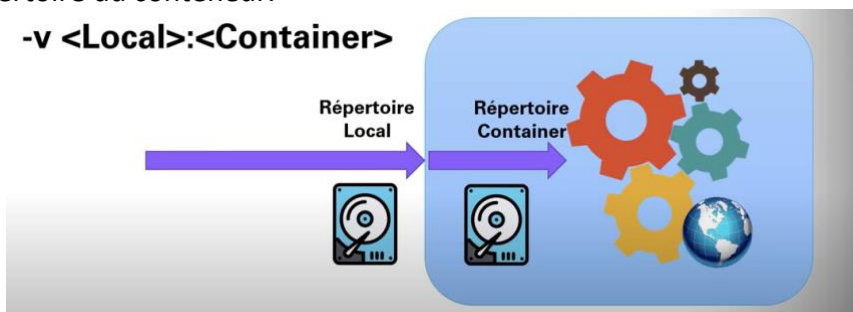
Donc, on peut les voir comme des portes à ouvrir pour accéder au conteneur. Ainsi, on peut disposer de plusieurs types de mapping.

Mapping de port :



Ici, nous demandons à Docker de rediriger tout ce qui arrive du port 80 de l'extérieur du conteneur vers le port 80 du serveur web installé à l'intérieur du conteneur.

On utilisera également le mapping de répertoire pour accéder au disque du conteneur. L'option **-v** permet d'établir un lien entre un répertoire de notre ordinateur physique et un répertoire du conteneur.



Pour utiliser le serveur mysql

mysql -u root -p

Passwd :

